RELATÓRIO DO PROJETO

GERENCIAMENTO DE PROCESSOS

Disciplina: Sistemas Operacionais

Professor: Clóvis Ferraro

Grupo: 07

SUMÁRIO

1 INTRODUÇÃO 3

1.1 Processos e Thread.....3

1.2 O que é um gerenciamento de processos.....3

1.3 Sistema operacional e escalonamento.....3

2 METODOLOGIA.....4

1.1 Sistemas operacionais utilizados.....4

1.2 Ferramentas utilizadas.....4

3. ANÁLISE DOS COMANDOS E CONFIGURAÇÕES.....4

3.1 Linux ps.....4

3.2 Windows (Gerenciador de Tarefas) 6

3.3 Android.....7

4. COMPARAÇÃO E ANÁLISE CRÍTICA.....10

4.1 Tabela Comparativa.....11

4.2 Linux.....11

4.3 Windows.....11

4.4 Android.....11

5. CONCLUSÃO.....12

5.1 Aprendizados.....12

5.2 Resposta ao Objetivo.....12

6. AUTOAVALIAÇÃO.....12

1 INTRODUÇÃO

O objetivo deste relatório é demonstrar de forma teórica e prática, os principais comandos e conceitos usados no gerenciamento de processos nos sistemas operacionais Linux, Windows e Android. O gerenciamento de processos é essencial para o bom funcionamento de qualquer sistema operacional, permitindo assim o usuário acompanhar, melhorar e controlar o uso dos recursos do seu computador. O escalonador organiza a execução dos processos e threads, garantindo melhor aproveitamento da CPU e eficiência do sistema.

1.1 Processos e Thread

O processo se trata do programa que está sendo executado naquele momento.

Um processo pode conter o código do programa, seu estado atual (representado pelo contador de programas), as pilhas, os registradores e um conjunto de informações de gerenciamento.

Um thread é uma sequência de instruções que pode ser executada simultaneamente com outras threads do mesmo processo, utilizando o mesmo espaço de memória e recursos. Essa estrutura permite que um único processo realize várias tarefas ao mesmo tempo, tornando a execução mais rápida eficiente.

1.2 O que é um gerenciamento de processo?

O gerenciamento de processos garante que o sistema operacional saiba quem está sendo executado, por quanto tempo e com quais recursos, mantendo tudo funcionando de maneira estável e eficiente.

1.3 Sistema operacional e escalonamento

Os processos são gerenciados pelo sistema operacional através de um ciclo de vida, que classifica seus estados como: pronto, em execução e bloqueado.

Processos em estado de pronto estão esperando a sua vez para serem executados no processador, enquanto os processos em execução estão naquele momento utilizando a CPU.

Já processos em estado de bloqueado, estão esperando algum evento externo, como a conclusão de uma operação de E/S, para serem atribuídos novamente ao estado de pronto.

O sistema operacional usa o escalonador para controlar a passagem dos processos em seus diferentes estados. Esse escalonador e a threads são os responsáveis por definir a ordem e o tempo de CPU de cada processo, garantindo assim que o processador seja utilizado da forma mais eficiente, equilibrando desempenho, velocidade de resposta e uso adequado dos recursos do sistema.

2. METODOLOGIA

2.1 Sistemas operacionais utilizados

Os sistemas operacionais que foram utilizados para a realização deste relatório.

Windows 11, Linux Ubuntu 24.4 e Android x86.

2.2 Ferramentas utilizadas

Linux: ps.

Windows: gerenciador de tarefas.

Android: adb shell.

3. Análise dos comandos e configurações

3.1 Linux (ps)

O comando ps mostra um registro dos processos que estão rodando no sistema. Diferente do Gerenciador de Tarefas do Windows ele não atualiza em tempo real - é uma visão instantânea do momento.

```
guest-mm2x7k@support21: ~
guest-mm2x7k@support21:~$ ps
  PID TTY          TIME CMD
 5425 pts/2    00:00:00 bash
 5469 pts/2    00:00:00 ps
guest-mm2x7k@support21:~$
```

Quando usado sozinho, o comando exibe quatro informações principais: **PID** (Identificador do processo), **TTY** (Terminal), **TIME** (Tempo de execução) e **CMD** (Comando que iniciou o processo).

Para ver mais detalhes, use **ps aux:**

• **a** mostra o processo de todos os usuários;

• **u** adiciona dados como uso de CPU, memória e dono do processo;

• **x** lista processos que não estão ligados a um terminal.

```
guest-mm2x7k@support21: ~
guest-mm2x7k@support21:~$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.0 185352  5984 ?        Ss   08:39   0:01 [systemd]
root         2  0.0  0.0      0     0 ?        S    08:39   0:00 [kthreadd]
root         4  0.0  0.0      0     0 ?        I<   08:39   0:00 [kworker/0:0H]
root         5  0.0  0.0      0     0 ?        I    08:39   0:00 [kworker/u8:0]
root         6  0.0  0.0      0     0 ?        I<   08:39   0:00 [mm_percpu_wq]
root         7  0.0  0.0      0     0 ?        S    08:39   0:00 [ksoftirqd/0]
root         8  0.1  0.0      0     0 ?        I    08:39   0:02 [rcu_sched]
root         9  0.0  0.0      0     0 ?        I    08:39   0:00 [rcu_bh]
root        10  0.0  0.0      0     0 ?        S    08:39   0:00 [migration/0]
root        11  0.0  0.0      0     0 ?        S    08:39   0:00 [watchdog/0]
root        12  0.0  0.0      0     0 ?        S    08:39   0:00 [cpuhp/0]
root        13  0.0  0.0      0     0 ?        S    08:39   0:00 [cpuhp/1]
root        14  0.0  0.0      0     0 ?        S    08:39   0:00 [watchdog/1]
root        15  0.0  0.0      0     0 ?        S    08:39   0:00 [migration/1]
root        16  0.0  0.0      0     0 ?        S    08:39   0:00 [ksoftirqd/1]
root        18  0.0  0.0      0     0 ?        I<   08:39   0:00 [kworker/1:0H]
root        19  0.0  0.0      0     0 ?        S    08:39   0:00 [cpuhp/2]
root        20  0.0  0.0      0     0 ?        S    08:39   0:00 [watchdog/2]
root        21  0.0  0.0      0     0 ?        S    08:39   0:00 [migration/2]
root        22  0.0  0.0      0     0 ?        S    08:39   0:00 [ksoftirqd/2]
root        24  0.0  0.0      0     0 ?        I<   08:39   0:00 [kworker/2:0H]
root        25  0.0  0.0      0     0 ?        S    08:39   0:00 [cpuhp/3]
```

Se você quiser ver a hierarquia dos processos (pais e filhos), basta usar **ps-axjf.**

```
guest-mm2x7k@support21: ~
guest-mm2x7k@support21:~$ ps -axjf
 PPID   PID  PGID   SID TTY      TPGID STAT   UID   TIME COMMAND
    0     2     0     0 ?           -1 S        0   0:00 [kthreadd]
    2     3     0     0 ?           -1 I        0   0:00  \_ [kworker/0:0]
    2     4     0     0 ?           -1 I<       0   0:00  \_ [kworker/0:0H]
    2     5     0     0 ?           -1 I        0   0:00  \_ [kworker/u8:0]
    2     6     0     0 ?           -1 I<       0   0:00  \_ [mm_percpu_wq]
    2     7     0     0 ?           -1 S        0   0:00  \_ [ksoftirqd/0]
    2     8     0     0 ?           -1 I        0   0:01  \_ [rcu_sched]
    2     9     0     0 ?           -1 I        0   0:00  \_ [rcu_bh]
    2    10     0     0 ?           -1 S        0   0:00  \_ [migration/0]
    2    11     0     0 ?           -1 S        0   0:00  \_ [watchdog/0]
    2    12     0     0 ?           -1 S        0   0:00  \_ [cpuhp/0]
    2    13     0     0 ?           -1 S        0   0:00  \_ [cpuhp/1]
    2    14     0     0 ?           -1 S        0   0:00  \_ [watchdog/1]
    2    15     0     0 ?           -1 S        0   0:00  \_ [migration/1]
    2    16     0     0 ?           -1 S        0   0:00  \_ [ksoftirqd/1]
    2    18     0     0 ?           -1 I<       0   0:00  \_ [kworker/1:0H]
    2    19     0     0 ?           -1 S        0   0:00  \_ [cpuhp/2]
    2    20     0     0 ?           -1 S        0   0:00  \_ [watchdog/2]
    2    21     0     0 ?           -1 S        0   0:00  \_ [migration/2]
    2    22     0     0 ?           -1 S        0   0:00  \_ [ksoftirqd/2]
    2    24     0     0 ?           -1 I<       0   0:00  \_ [kworker/2:0H]
    2    25     0     0 ?           -1 S        0   0:00  \_ [cpuhp/3]
```

Além dessas opções, aqui estão outros exemplos úteis de comandos ps:

• **ps-u** (nome de usuário) mostra processos de um usuário específico.

• **ps-e ou ps-A** Lista todos os processos ativos.

• **ps-T** exibe todos os processos ligados ao terminal.

• **ps-C** (nome) filtra os processos pelo nome e mostra também os processos filhos.

3.2 Windows (Gerenciador de Tarefas)

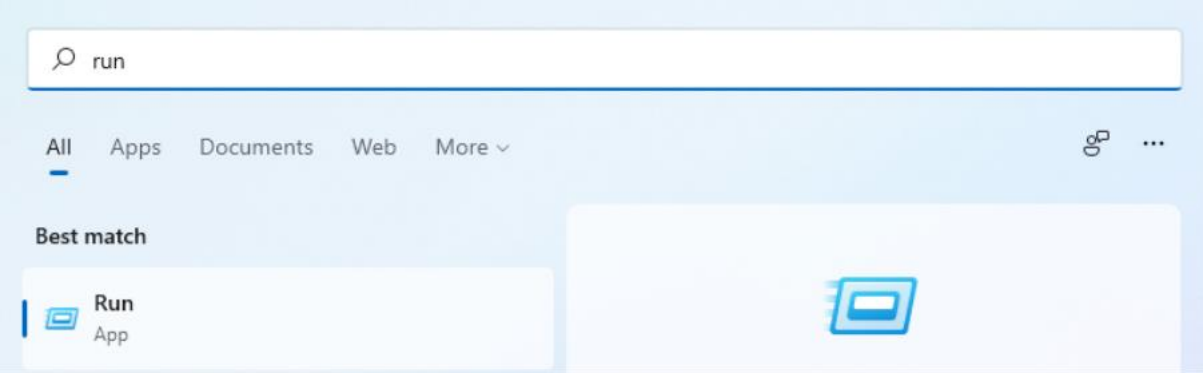
No Windows 11, o **Gerenciador de Tarefas** pode ser aberto de diversas maneiras: pressionando **Ctrl + Alt + Delete, Ctrl + Shift + Esc**, **Windows + X**, ou pelo comando **executar** (comando que será utilizado para elaboração deste tópico)

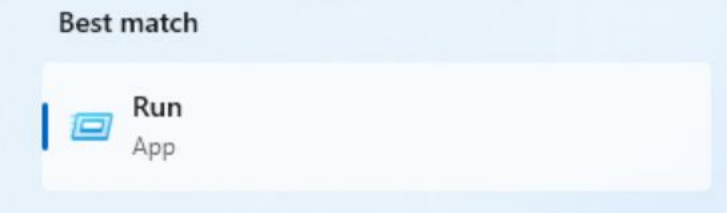
Comando Executar

• Clique no menu iniciar;

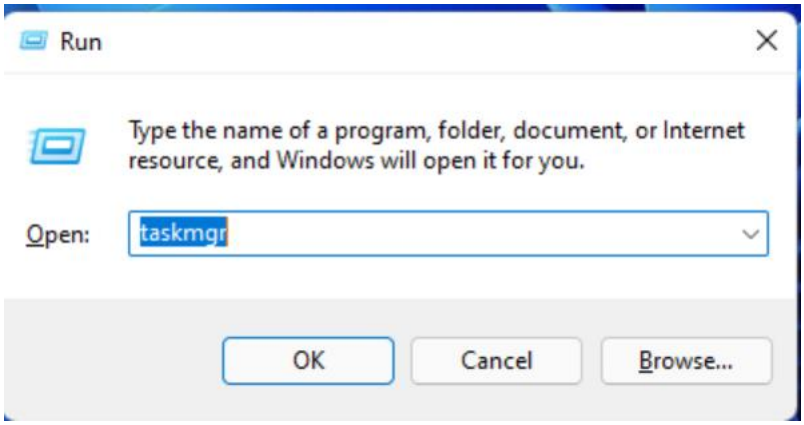
*Uma alternativa para iniciar esse comando é usando o teclado, pressione Windows e R simultaneamente, assim que o comando executar for aberto, solte as teclas.

• Quando o menu Iniciar for aberto, digite Run → Best Match.





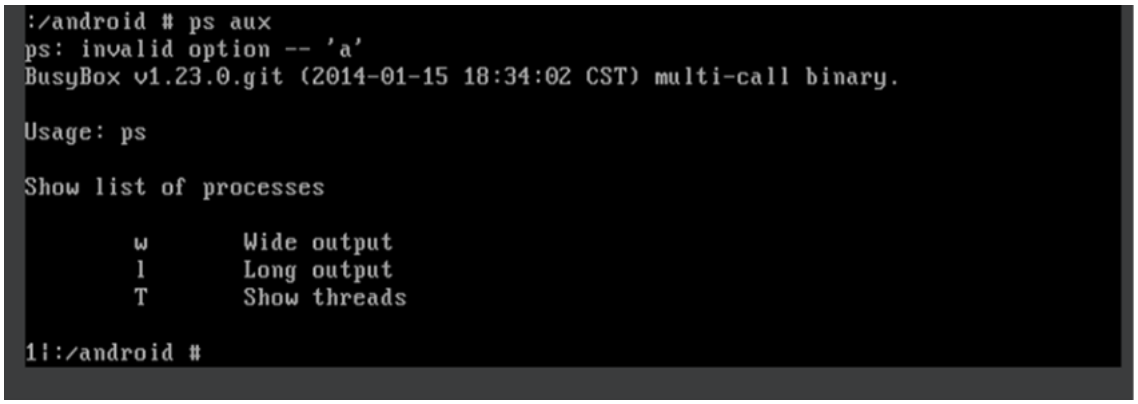
• Digite taskmgr → OK.



• O Gerenciador de Tarefas será iniciado.

3.3 Android

Ao usar o comando “ps aux”, o terminal exibe diferentes formas de visualizar os processos do sistema (Wide output, Long output e Show threads). Neste relatório, apenas o modo Long output funcionou, mostrando a lista de processos em execução no sistema.



```
        w       Wide output
        l       Long output
        T       Show threads

1¦:/android # w
system/bin/sh: w: not found
127¦:/android # l
acct                        init.superuser.rc          sbin
bin                         init.usb.configfs.rc       sdcard
bugreports                  init.usb.rc                sepolicy
cache                       init.zygote32.rc           storage
charger                     init.zygote64_32.rc        sys
config                      lib                        system
d                           mnt                        ueventd.android_x86_64.rc
data                        odm                        ueventd.rc
default.prop                oem                        vendor
dev                         plat_file_contexts         vendor_file_contexts
etc                         plat_hwservice_contexts    vendor_hwservice_contexts
fstab.android_x86_64        plat_property_contexts     vendor_property_contexts
init                        plat_seapp_contexts        vendor_seapp_contexts
init.android_x86_64.rc      plat_service_contexts      vendor_service_contexts
init.environ.rc             proc                       vndservice_contexts
init.rc                     product
:/android # _
```

O comando **top** permite acompanhar em tempo real os processos ativos, exibindo informação como o **PID** (Identificador de processo), que será necessário para comandos posteriores.

```
Mem: 89584K used, 3947296K free, 0K shrd, 135072968K buff, 135073016K cached
CPU:  0.0% usr  0.0% sys  0.0% nic 99.9% idle  0.0% io  0.0% irq  0.0% sirq
Load average: 0.00 0.00 0.00 1/69 2165
  PID  PPID USER     STAT   VSZ %VSZ CPU %CPU COMMAND
 1054     1 0        S     8324  0.2   1  0.0 system/bin/sh -l
 2165  1054 0        R     2528  0.0   2  0.0 top
  986     1 0        S     2528  0.0   0  0.0 /bin/sh
  992     1 0        S     2528  0.0   1  0.0 /bin/sh
    1     0 0        S     2524  0.0   0  0.0 /bin/sh /init
   22     2 0        SW       0  0.0   2  0.0 [migration/2]
   17     2 0        SW       0  0.0   1  0.0 [migration/1]
 1020     2 0        SW<      0  0.0   2  0.0 [loop1]
   10     2 0        IW       0  0.0   0  0.0 [rcu_preempt]
 1016     2 0        SW<      0  0.0   0  0.0 [loop0]
   14     2 0        IW       0  0.0   0  0.0 [kworker/0:1-eve]
   18     2 0        SW       0  0.0   1  0.0 [ksoftirqd/1]
   36     2 0        IW       0  0.0   1  0.0 [kworker/u6:2-ev]
 1038     2 0        IW       0  0.0   1  0.0 [kworker/1:2-eve]
    2     0 0        SW       0  0.0   0  0.0 [kthreadd]
    3     2 0        IW<      0  0.0   0  0.0 [rcu_gp]
    4     2 0        IW<      0  0.0   0  0.0 [rcu_par_gp]
    5     2 0        IW       0  0.0   0  0.0 [kworker/0:0-eve]
    6     2 0        IW<      0  0.0   0  0.0 [kworker/0:0H-kb]
    7     2 0        IW       0  0.0   0  0.0 [kworker/u6:0-ev]
    8     2 0        IW<      0  0.0   0  0.0 [mm_percpu_wq]
```

Antes de trabalhar com o PID, podemos alterar a prioridade de execução de um script usando o comando “nice-n 10”, que reduz a prioridade do processo.

```
nice -n 10
BusyBox v1.23.0.git (2014-01-15 18:34:02 CST) multi-call binary.

Usage: nice [-n ADJUST] [PROG ARGS]

Change scheduling priority, run PROG

        -n ADJUST       Adjust priority by ADJUST

1|:/android # nice -n 10
BusyBox v1.23.0.git (2014-01-15 18:34:02 CST) multi-call binary.

Usage: nice [-n ADJUST] [PROG ARGS]

Change scheduling priority, run PROG

        -n ADJUST       Adjust priority by ADJUST

1|:/android # _
```

Com o PID em mãos, é possível:

• usar “renice 15-p" para mudar a prioridade de um processo já em execução;

• usar “kill” para encerrar um processo de forma normal (SIGTERM);

• usar “kill-9” para forçar o encerramento de um processo (SIGKILL).

```
Mem: 79016K used, 3957896K free, 0K shrd, 135072968K buff, 135073016K cached
CPU:  0.0% usr  0.0% sys  0.0% nic  100% idle  0.0% io  0.0% irq  0.0% sirq
Load average: 0.00 0.00 0.00 1/69 2170
  PID  PPID USER     STAT   VSZ %VSZ CPU %CPU COMMAND
  958     2 0        IW<      0  0.0   2  0.0 [kworker/u7:0]
  975     2 0        IW       0  0.0   2  0.0 [kworker/2:2]
 1001     2 0        IW<      0  0.0   0  0.0 [kworker/0:1H-kb]
 1007     2 0        IW<      0  0.0   1  0.0 [kworker/1:1H-kb]
 1009     2 0        IW<      0  0.0   2  0.0 [kworker/2:1H-kb]
 1010     2 0        SW       0  0.0   0  0.0 [jbd2/sda1-8]
 1011     2 0        IW<      0  0.0   0  0.0 [ext4-rsv-conver]
 1022     2 0        IW<      0  0.0   0  0.0 [ext4-rsv-conver]
 2156     2 0        IW       0  0.0   0  0.0 [kworker/0:2-eve]
:/android # renice 15 -p 2156
:/android # kill 1022
:/android # kill -9 1011
```

4. COMPARAÇÃO E ANÁLISE CRÍTICA

4.1 Tabela Comparativa: A principal diferença é que o Linux gerencia processos com base na prioridade nice, um valor ajustável que determina a "cortesia" de um processo (valores mais baixos = maior prioridade). O Windows gerencia prioridades de forma mais dinâmica, alocando recursos de forma mais rígida e integrada ao sistema, enquanto o Ciclo de Vida do Aplicativo Android se refere ao gerenciamento de recursos (tempo de execução, memória) e ao estado dos processos do aplicativo para otimizar o desempenho e a vida da bateria, e não diretamente à prioridade de CPU.

4.2 Linux (Prioridade *Nice*)

- **Filosofia**: O kernel do Linux gerencia as prioridades automaticamente, mas permite que os usuários interfiram através do valor "nice".

- **Controle**:
  - O comando `nice` define a prioridade de um novo processo.
  - O comando `renice` altera a prioridade de um processo em execução.

4.3 Windows

- **Filosofia**: O Windows utiliza um sistema de prioridade mais integrado e dinâmico para gerenciar processos.

- **Controle**: Os usuários e o sistema controlam a prioridade em tempo real, e a preempção garante que processos mais importantes interrompam os menos importantes.

4.4 Android

- **Filosofia**: O ciclo de vida do aplicativo Android se concentra em gerenciar os processos do aplicativo (instâncias) para otimizar o uso de recursos, como CPU e memória, e prolongar a vida da bateria.

- **Controle**: O sistema operacional Android gerencia o ciclo de vida dos aplicativos de forma automática, usando diferentes mecanismos para otimizar o uso de recursos.

5. CONCLUSÃO

5.1 Aprendizados

Os processos são programas em execução que tem a própria memoria, e threads são partes menores dentro de um processo, os threads são capazes de deixar o sistema rápido e eficiente deixando que várias tarefas ocorram ao mesmo tempo dentro de um processo, escalonamento e as prioridades decidem qual tarefa do processo vai se executar primeiro. Os diferentes sistemas que presenciamos nessa tarefa são distintos já que se diferenciam em suas dinâmicas operacionais, **o Windows**, o escalonamento é baseado em **prioridades dinâmicas**, ou seja, o sistema ajusta automaticamente a prioridade das tarefas para manter o computador rápido e responsivo. No Linux o escalonamento é flexível e eficiente, fazendo com que o processador dívida o tempo no sistema para que e ele não se sobrecarregue.

O Android, que é baseado no Linux, o escalonamento segue a mesma ideia, mas é otimizado para economia de energia e desempenho em dispositivos móveis, priorizando tarefas que afetam diretamente o uso do usuário, como aplicativos abertos na tela.

5.2 RESPOSTA AO OBJETIVO

Ao longo do processo de realização do nosso relatório, foram identificadas dificuldades no sistema Android, sendo o mais desafiador por ter poucos comandos, mesmo com dificuldades conseguimos trazer comandos e realizar os mesmos, o Windows foi o mais simplório no quesito dificuldade, e sendo o mais agradável. Linux possui suas dificuldades, porém respondeu aos comandos com perfeição e facilidade, basta compreender o sistema operacional e usá-lo da melhor forma, pela visão do grupo realizamos o mesmo com facilidade e liberdade de comandos

6.0 AUTOAVALIAÇÃO

O grupo obteve um ótimo desenvolvimento, e todos agregando ao trabalho de forma harmoniosa e conjunta, tivemos dificuldades no sistema operacional Android, porém nada muito complicado, conseguimos nos desenvolver e entender toda a dinâmica do trabalho.

REFERENCIAS

1. BLOG GRAN CURSOS ONLINE. Sistemas operacionais: gerenciamento de processos. Disponível em: https://blog.grancursosonline.com.br/sistemas-operacionais-gerenciamento-de-processos/. Acesso em: 18 out. 2025.

2. PGSSC OGNA. Processos e Threads [PDF]. Disponível em: https://repositorio.pgsscogna.com.br/bitstream/123456789/65147/3/Processos%20e%20Threads.pdf. Acesso em: 18 out. 2025.

3. SANTANA, Bruno. Como gerenciar e listar processos no Linux usando os comandos ps, top, htop e atop. Hostinger, 09 abr. 2025. Disponível em: https://www.hostinger.com/br/tutoriais/como-listar-processos-linux. Acesso em: 18 out. 2025. ￼

4. DELL SUPPORT. Como iniciar o Gerenciador de Tarefas no Windows 11. Disponível em: https://www.dell.com/support/kbdoc/pt-pt/000192107/como-iniciar-o-gerenciador-de-tarefas-no-windows-11. Acesso em: 18 out. 2025.